NUMERICAL ANALYSIS
GUIDED LAB 3

ROB TOMPKINS

Note, all the files associated with Guided Lab 3 are checked into the :
https://github.com/chtompki/MathSpring2026/tree/main/NumericalAnalysis/GuidedLab3

Problems 1, 2, and 3, were predominantly only writing code. The code can be found at: naturalCubicSplines.m. Though, the code for these three problems also follows as

```
function [A,rvec,c,a,b,d] = naturalCubicSplines(x,y)
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % Natural cubic spline interpolator
    %
    % Inputs:
    %   x     - vector of x data points
    %   y     - vector of y data points
    %   xstar - point(s) where spline is to be evaluated
    %
    % Output:
    %   ystar - spline value(s) at xstar
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    if length(x) < 2 | length(y) < 2
        error('Input vectors are not long enough, please make them of length two or more and be of the same size')
    end
    if length(x) ~= length(y)
        error('Input vectors are not the same length')
    end

    % Make sure x and y are column vectors
    x = x(:);
    y = y(:);

    % n = number of subintervals
    n = length(x) - 1;

    % Initialize A and rvec
    A = zeros(n+1,n+1);
    rvec = zeros(n+1,1);

    % Calculate vector of h values
    h = diff(x);

    % Natural spline boundary conditions
    A(1,1) = 1;
    A(n+1,n+1) = 1;
```

Date: March 16, 2026.

```
    % Populate rest of matrix A and RHS vector
    for i = 2:n
        A(i,i-1) = h(i-1);
        A(i,i)   = 2*(h(i-1) + h(i));
        A(i,i+1) = h(i);
        rvec(i)  = 3*((y(i+1)-y(i))/h(i) - (y(i)-y(i-1))/h(i-1));
    end

    % Solve for c coefficients
    c = A \ rvec;

    % Compute a coefficients
    a = zeros(n+1,1);
    for j = 1:n+1
        a(j) = y(j);
    end

    % Compute b and d coefficients
    b = zeros(n,1);
    d = zeros(n,1);

    for j = 1:n
        d(j) = (c(j+1) - c(j)) / (3*h(j));
        b(j) = (a(j+1) - a(j))/h(j) - (h(j)/3)*(2*c(j) + c(j+1));
    end
end
```

Problem 4 works with the following code:

```
coeffs=zeros(length(x)-1, 4);
for k=1:length(x)-1
    coeffs(k,:)=[d(k) c(k) b(k) a(k)];
end

breaks=[x];
pp=mkpp(breaks,coeffs);
ystar=ppval(pp,xstar);
```

At first, we're running `pp=mkpp(breaks,coeffs)` which takes the intervals over the x vector, and the coefficients, and builds cubic polynomials using the values d_i, c_i, b_i, and a_i for the interval between x_i and x_{i+1}. Running `mkpp` results in the output `pp` is a matlab struct with the following fields: `form`, `breaks`, `coefs`, `pieces`, `order`, and `dim`. `form` seems to be a constant string term with the value *pp* for piecewise polynomial. `breaks` represents the increasing list of x values of our points. `coefs` is a matrix of coefficients set up for the equation

$$f(x) = a(x - x_i)^3 + b(x - x_i)^2 + c(x - x_i) + d$$

Molality	0.005	0.010	0.020	0.050	0.100	0.200	0.500	1.000	2.000
Coefficient	0.924	0.896	0.859	0.794	0.732	0.656	0.536	0.430	0.316

TABLE 1. The Molality Given the Meah Activity Coefficients of Silver Nitrate

Days After Birth	0	6	10	13	17	20	28
avg. weight, *mg* (from young leaves)	7	18	45	40	32	30	30.5
avg. weight, *mg* (from mature leaves)	7	16	19	15	12	10.5	10

TABLE 2. Average weights of larvae from Feeny, P. (1970) *Ecology*, **51** (4), 565-581.

based on the i^{th} interval. `pieces` is the number of elements in the partition; `order` is the order of the polynomials; and `dim` is the dimensionality of the target. All of this then in turn gets sent to the function `ppeval(pp,xstar)`, which is a canned matlab function specifically for taking this data structure pp and generating y values given an arbitrary collection of x values. We were told to not edit this code, but curiously `xstar` is not defined, so we must in some fashion by defining it. It seems the best thing to do here is to take the minimum and maximum values from x and evenly subdivide the system for evaluation and to get a plot.

Problem 4 above adapted the original code from above such that the signature of the `naturalCubicSplines` function follows as `ystar=naturalCubicSplines(x,y,xstar)`. This is quite important for Problem 5 because we run the data in Table 1 through the method. We then set x to be Molality, and y to be Coefficient and $xstar = 0.032$ to see what our method outputs. We get $ystar = 0.8283$. The following code:

```
x = [0.005 0.010 0.020 0.050 0.100 0.200 0.500 1.000 2.000];
y = [0.924 0.896 0.859 0.794 0.732 0.656 0.536 0.430 0.316];

ystar = naturalCubicSpline1(x,y,0.032)
```

yields the following output:

```
>> gl3prob5

ystar =

    0.8283
```

Lastly note we have all this in thge file gl3prob5.m.

For problem 6 we consider Table 2 over which we will compute cubic spline interpolants. For portion (a) of Problem 6, we have the following code and output

```
daysAfterBirth  = [0 6 10 13 17 20 28];
weightYoungLeaves = [7 18 45 40 32 30.5 30];
weightMatureLeaves = [7 16 19 15 12 10.5 10];

youngWeight = naturalCubicSpline1(daysAfterBirth,weightYoungLeaves,3)
matureWeight = naturalCubicSpline1(daysAfterBirth,weightMatureLeaves,3)
meanWeight = (youngWeight + matureWeight)/2

youngWeightMatlab = spline(daysAfterBirth,weightYoungLeaves,3)
```

```
matureWeightMatlab = spline(daysAfterBirth,weightMatureLeaves,3)
meanWeightMatlab = (youngWeightMatlab + matureWeightMatlab)/2
```

So our mean weight works out to be $9.3754mg$ for our not-a-knot cubic spline while the matlab cubic spline outputs $3.8707mg$.

For portion (b) of problem 6 the fastest method for getting to where the maximum of our natural cubic splines lies in simply plotting and looking at the plots for the maximum, so we went about that. Our plot follows in Figure 1. In the figure we've plotted all three, the young leaf natural cubic spline, the mature leaf cubic spline, and the mean cubic spline. We find that in the plot the maximum for the mean cubic spline happens at 10.5 days at a weight of 32.1432. The hypothesis that tannins in the mature leaves inhibit growth seems reasonable because of the discrepancy between the plot of the solid red and the solid black lines in Figure **??**. Clearly the younger leaves have more mass.

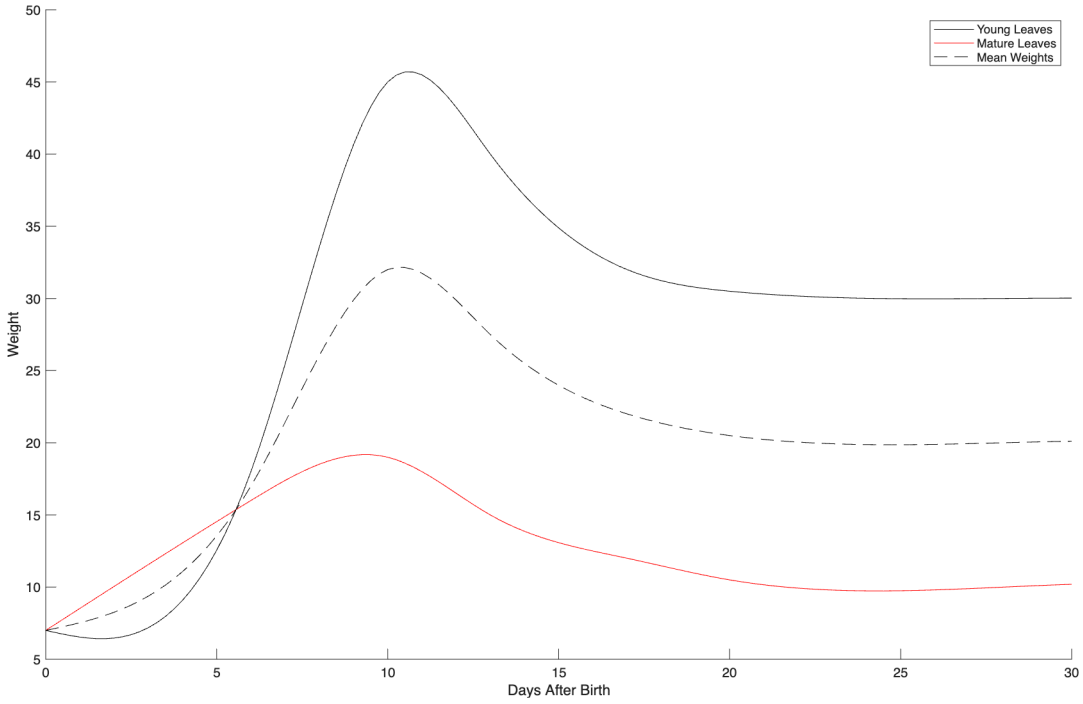

FIGURE 1. Weight of Young and Mature Leaves, in mg

.

For problems 7 and 8 we found a dataset of temperature over time, which is a continuously moving aspect with respec to time. The data came from a book, namely "Numerical Methods Using Matlab" by John Matthews and Kurtis Fink pp. 279. So a cubic spline would be a perfect mechanism for interpolating between our discrete measurements of temperature to try to create a function that will give us temperature at any given time (approximately). The data isn't particularly interesting other than it is a continuous function in real life, so

Time, p.m.	Degrees	Time, a.m.	Degrees
1	66	1	58
2	66	2	58
3	65	3	58
4	64	4	58
5	63	5	57
6	63	6	57
7	62	7	57
8	61	8	58
9	60	9	60
10	60	10	64
11	59	11	67
Midnight	58	Noon	68

TABLE 3. Temperature in Farenheit over a day

our cubic spline will probably be a good representation of where the temperature is during the day generally. The data set is given in Table 3

The problem calls for a for loop to appropriately generate our `xstar` vector to be passed into the function which haves us dividing 24 into 100 equal segments of 0.24 hours (I suppose something not quite 15 minutes). The for loop in question should look like this:

```
xstar=zeros(100);
for i = 1:100
    xstar(i) = i*0.24;
end
```

Note we could have declared this at `xstar=0.24:0.24:24`, nonetheless we move on with the `for` loop. The code in its entirety looks like:

```
time=[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24];
temp=[58 58 58 58 57 57 57 58 60 64 67 68 66 66 65 64 63 63 62 61 60 60 59 58];

xstar = zeros(100);
for i = 1:100
    xstar(i) = i*0.24;
end

ystar = naturalCubicSplines(time,temp,xstar);

figure;
hold on;
plot(xstar,ystar, 'k-');
plot(time,temp, 'ko');
xlabel('Time (hours)');
ylabel('Temperature (°F)');
legend('Cubic Spline', 'Data Points');
hold off;
```

And the plot follows in Figure 2.

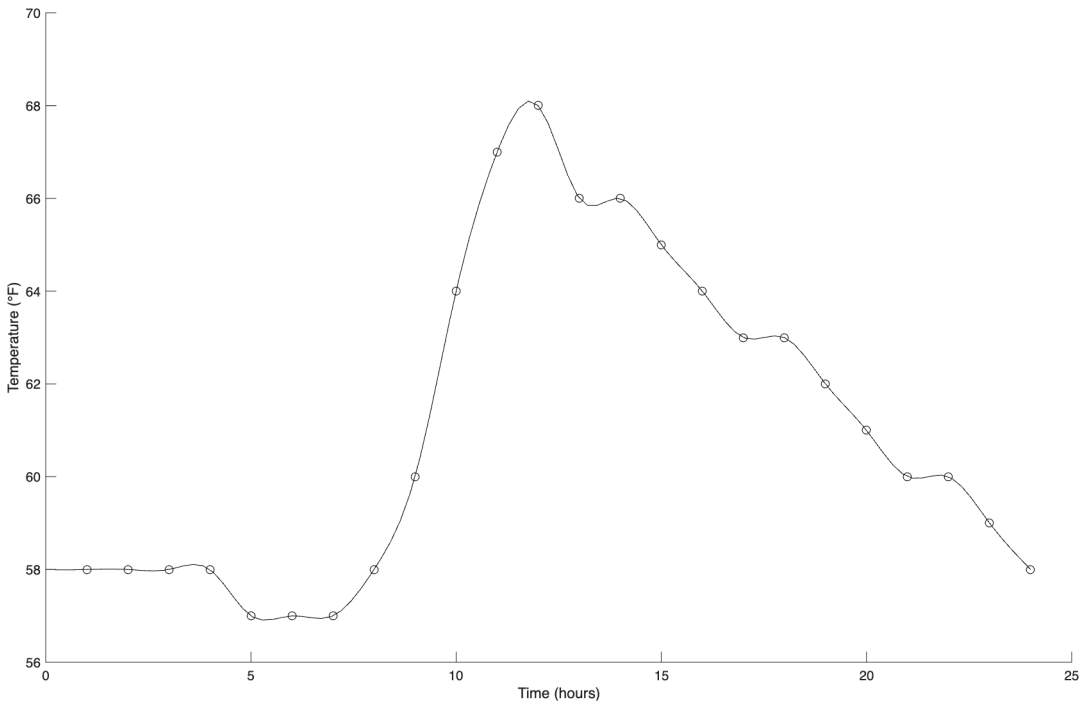

FIGURE 2. Temperature in Fahrenheit for a given day in a random location interpolated.

.

Regarding problem 8, we have gone through and removed the points as requested. The code follows as:

```
time=[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24];
temp=[58 58 58 58 57 57 57 58 60 64 67 68 66 66 65 64 63 63 62 61 60 60 59 58];
time2=[ 2 4 6 8 10 12 14 16 18 20 22 24];
temp2=[58 58 57 58 64 68 66 64 63 61 60 58];

xstar = zeros(100);
for i = 1:100
    xstar(i) = i*0.24;
end

ystar = naturalCubicSplines(time,temp,xstar);
ystar2 = naturalCubicSplines(time2,temp2,xstar);

figure;
hold on;
plot(xstar,ystar, 'k-');
plot(xstar,ystar2, 'k.');
plot(time,temp, 'ko');
xlabel('Time (hours)');
ylabel('Temperature (°F)');
hold off;
```

Our plot is given in Figure 3.

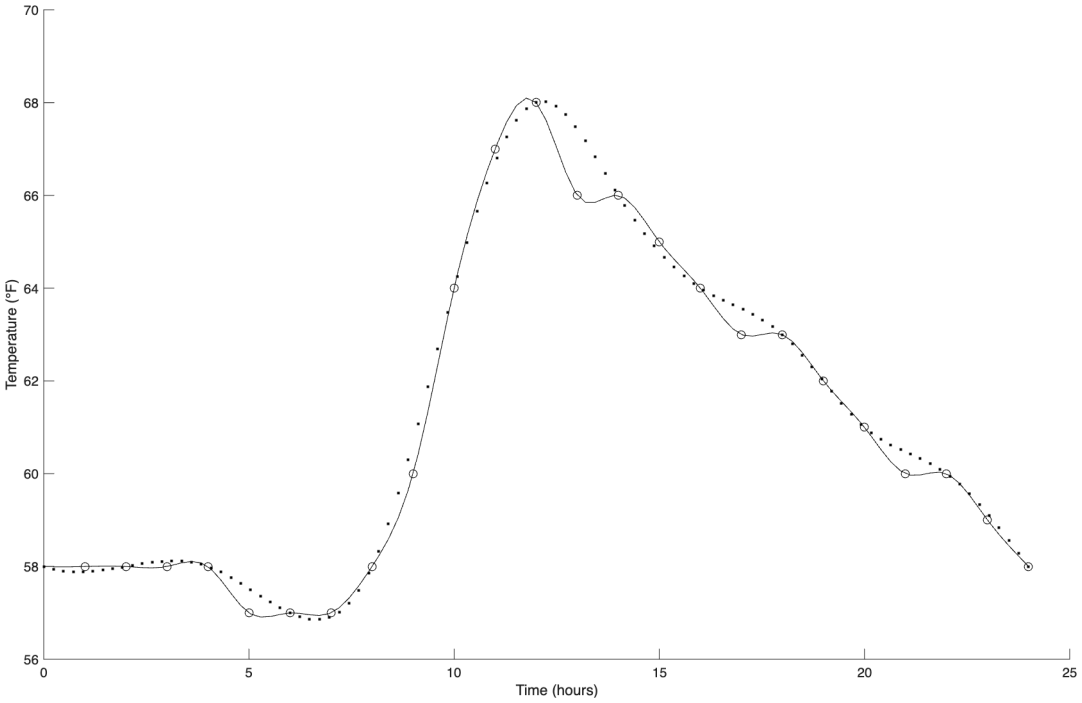

FIGURE 3. Temperature in Fahrenheit for a given day in a random location interpolated on the hour and the even hour.

.

What can be determined by the dropping of points regarding interpolation? Well certainly we get different plots, the functions diverge as there is more room for wiggle with the spline on fewer points. The graphs are both continuous, but clearly the more points in our data set the closer our curve is going to be to the actual temperature of the day, and that is a limit process.